

La convención de nombres de Spring Data JPA (conocida como **Query Methods**) es una de las herramientas más potentes porque permite crear consultas SQL complejas simplemente nombrando los métodos en la interfaz del repositorio.

1. Estructura Básica

El nombre del método se compone de:

[Sujeto] + [Predicado] + [Criterio] + [Modificador]

- **Sujeto:** find...By, exists...By, count...By, delete...By.
- **Predicado/Criterio:** El nombre del atributo en tu clase Paciente (ej: Nombres, Run).
- **Modificador:** Asc, Desc, All, First.

2. Ejemplos Sin Parámetros

Se usan generalmente para conteos o listas globales con orden específico.

Requerimiento	Nombre del Método en Repository	Explicación
Contar total	long count();	Heredado de JPA, cuenta todos los registros.
Listar y ordenar	findAllOrderByNombresAsc();	Trae todos de la A a la Z.
Traer el primero	findFirstOrderByFechaNacimientoAsc();	Trae al paciente más anciano de la tabla.

3. Ejemplos Con Parámetros (Filtros)

Aquí es donde pasas valores para que JPA cree el WHERE de la consulta.

Filtro	Nombre del Método	Uso (Parámetro)
Búsqueda exacta	<code>findByRun(String run);</code>	Busca al paciente que coincida exactamente con ese RUN.
Coincidencia parcial	<code>findByApellidosContaining(String parte);</code>	Como un LIKE %valor%. Ideal para buscadores.
Rango de fechas	<code>findByFechaNacimientoBetween(LocalDate ini, LocalDate fin);</code>	Trae pacientes nacidos entre dos fechas.
Lógica AND	<code>findByNombresAndApellidos(String n, String a);</code>	Deben coincidir ambos campos.
Lógica OR	<code>findByNombresOrApellidos(String texto, String texto2);</code>	Basta con que coincida uno.
Ignorar Mayúsculas	<code>findByNombresIgnoreCase(String nombre);</code>	Busca "JUAN", "juan" o "Juan" por igual.

4. Ejemplos con Operadores Lógicos y Comparación

Java

```
// Pacientes mayores a una fecha (Más jóvenes que...)
List<Paciente> findByFechaNacimientoAfter(LocalDate fecha);

// Pacientes con un correo específico (existe o no existe)
boolean existsByCorreo(String correo);
// Buscar pacientes cuyo apellido empiece con una letra
List<Paciente> findByApellidosStartingWith(String letra);
```

💡 Reglas de Oro:

1. **CamelCase:** Los nombres de los métodos deben seguir estrictamente el CamelCase. Si el atributo es fechaNacimiento, el método debe decir ...ByFechaNacimiento... (la F y la N en mayúscula).
2. **Tipo de Retorno:**
 - Si esperas muchos: List<Paciente>.
 - Si esperas uno solo: Optional<Paciente> o Paciente.
 - Si es una verificación: boolean.
 - Si es un conteo: long o int.
3. **Orden de Parámetros:** Los parámetros del método deben ir en el mismo orden en que aparecen en el nombre del método.
 - Ej: findByNombresAndCorreo(String nombre, String correo) -> **Correcto.**

Para cerrar el ciclo de aprendizaje de la **Evaluación 1**, vamos a implementar el filtro de **Búsqueda por RUN**. Este es el ejemplo perfecto porque combina un filtro personalizado, el uso de Optional (buena práctica de Java) y el manejo de respuestas 404.

1. Repositorio (PacienteRepository.java)

Añadimos el método que buscará un paciente específico por su RUN. Como el RUN es único, esperamos que nos devuelva un solo objeto. Usamos Optional para manejar de forma elegante si no lo encuentra.

Java

```
@Repository
public interface PacienteRepository extends JpaRepository<Paciente, Long> {

    // Busca un paciente por su RUN exacto
    Optional<Paciente> findByRun(String run);
}
```

2. Servicio (PacienteService.java)

El servicio llamará al repositorio. Al devolver un Optional, le damos la libertad al controlador de decidir qué hacer si el resultado está vacío.

Java

```
public Paciente buscarPorRun(String run) {
    // .orElse(null) hace que si no lo encuentra, devuelva un nulo
    // Así mantenemos la lógica de los "if" en el controlador
    return repo.findByRun(run).orElse(null);
}
```

3. Controlador (PacienteController.java)

Creamos un endpoint que reciba el RUN como parámetro en la URL.

- **Ruta:** GET /api/v1/pacientes/buscar/{run}

Java

```
@GetMapping("/buscar/{run}")
public ResponseEntity<?> obtenerPorRun(@PathVariable String run) {
    Paciente paciente = pacienteService.buscarPorRun(run);

    if (paciente != null) {
        // Respuesta 200 OK si lo encuentra
        return ResponseEntity
            .status(HttpStatus.OK)
            .body(paciente);
    } else {
        // Respuesta 404 Not Found si el RUN no existe
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body("Error: No se encontró ningún paciente con el RUN: " +
run);
    }
}
```

 Resumen de conceptos:

1. **@PathVariable**: Se usa para capturar valores que vienen directamente en la ruta de la URL (como el RUN o el ID).
2. **Optional<T>**: Es un "contenedor" que puede o no tener un valor. Evita errores de puntero nulo (NullPointerException).
3. **Diferencia entre find y exists**:
 - existsByRun: Devuelve true/false. Útil para validar antes de crear.
 - findByRun: Devuelve el **objeto completo**. Útil para mostrar los datos en pantalla.

 Caso de prueba en Postman:

1. **Prueba Exitosa: GET**
http://localhost:8080/api/v1/pacientes/buscar/12345678-9
 - *Resultado*: JSON del paciente y Código 200.
2. **Prueba Fallida: GET** http://localhost:8080/api/v1/pacientes/buscar/999
 - *Resultado*: "Error: No se encontró..." y Código 404.